
Final Lab Assignment

What:

In this assignment you are to write a C or Fortran program that computes the Mandelbrot set using a hybrid parallel programming model (MPI+OpenMP):

1. Use an **OpenMP** and **MPI** hybrid code to parallelize the Mandelbrot set. Do not write two separate codes.
2. Create a **detailed PDF** file with a write-up that includes information about the hybrid algorithm that you used, see the Expected Result section below.
3. Leave the PDF file in the same directory where your assignment is.

Why:

Using a hybrid MPI+OpenMP approach to parallel and distributed computing is very common in a supercomputer environment. It allows for easy communication between nodes using MPI without sacrificing the ease of using OpenMP for node level parallelism. In addition, computing the Mandelbrot set presents a significant challenge due to the nature of the calculation.

How:

Typically, one uses a single MPI rank per node to facilitate communication between nodes and OpenMP to utilize all the cores on each node. Use this approach when solving this problem.

Because the result is a grayscale (or color) image of the Mandelbrot set, make sure you use the provided image tools C++ code that we used during the lecture (and assignment 4). This code will write the matrix that you calculate to an image that you can download and view locally.

Computing the Mandelbrot set in parallel requires extra care. Due to the nature of the calculation, there is a significant load balancing issue, i.e., some nodes/threads will complete their work before others. You must have fully load balanced program (across threads and ranks). There is more than one way to approach the load balancing for this application.

Use what you have learned this semester to achieve that fastest calculation possible, while ensuring that each node does approximately the same amount of work.

Expected Result:

In your **detailed** PDF writeup:

- Give a detailed description of what the Mandelbrot set is and how it is computed.
- Describe the algorithm that you used to solve the problem, including how you dealt with the load balancing.
- Include your final Mandelbrot set image.
- Create and include three plots in the writeup:
 - A strong scaling plot of the entire program (excluding the image writing). Use 6 or 12 OpenMP threads per node and vary the number of nodes to create the plot.
 - Produce a plot that shows that each rank takes the same amount of time to complete its part of the calculation. Essentially, do a run where you time each rank and display all of their results. **Their run times should be balanced.**
 - Produce a plot that shows that each thread on a single rank takes the same amount of time to complete its part of the calculation. **The thread times should be balanced.**